Panther Ludo - Iteration 1 - Class Diagram

	Package
V	view
C	controller
M	model
E	enumeration

MainFrame
-transition : MainFrame.FadeTransition
-current : String

+addScreen(name : String, screen : JComponent) : void
+showScreen(name : String) : void
+showScreen(name : String, animate : boolean) : void
+setOnSettings(handler : Runnable) : void
+setCornerButtonsVisible(visible : boolean) : void
+applyComponentOrientation(orientation : ComponentOrientation) : void
+setOnCloseRequested(handler : Runnable) : void

WelcomePanel
+setListener(listener : WelcomePanel.Listener) : void

MenuPanel
-news : MenuPanel.NewsSlideshow

+setNews(games : List<GameHistory.GameRecord>) : void
+setListener(listener : MenuPanel.Listener) : void
+showSession(admin : boolean, username : String) : void

SettingsPanel
+setListener(listener : SettingsPanel.Listener) : void

HistoryPanel
-rows : List<GameHistory.GameRecord>

+showRecords(records : GameHistory.ReadResult) : void
+showReadError() : void
+setListener(listener : HistoryPanel.Listener) : void

GameOverPanel
-levelChip : GameOverPanel.FactTile
-timeChip : GameOverPanel.FactTile
-playersChip : GameOverPanel.FactTile
-gameChip : GameOverPanel.FactTile
-saveChip : GameOverPanel.FactTile
-gameId : int
-stage : GameOverPanel.PodiumArea
-stats : GameOverPanel.StatsTable

+showSaved(gameId : int, saved : boolean) : void
+showQuit() : void
+showResult(game : Game.State) : void
+setListener(listener : GameOverPanel.Listener) : void

SetupPanel
-rows : List<SetupPanel.PlayerSetupRow>
-showAllProblems : boolean

+reset() : void
+setPlayableLevels(playable : Set<Level>, reason : String) : void
+setListener(listener : SetupPanel.Listener) : void

GamePanel
+NOTICE_DISPLAY_MILLIS : int
+TURN_BANNER_MILLIS : int
-board : BoardPanel
-dice : GamePanel.DiceView
-playerCard : GamePanel.PlayerCard
-formula : GamePanel.RollFormula
-eventLog : GamePanel.EventLog
-playerRows : List<GamePanel.RankingRow>
-selectable : Set<Piece>
-game : Game.State
-pendingPrompt : String
-timerEnabled : boolean
-computerPlaying : boolean
-lastTickSecond : int

+applyAppearance(effects : boolean) : void
+setListener(listener : GamePanel.Listener) : void
+bind(game : Game.State, gameId : int) : void
+unbind() : void
+showRules() : void
+isBusy() : boolean
+runWhenIdle(task : Runnable) : void
+showTurnTimer(timer : Game.TurnTimer) : void
+hideNotice() : void
+setComputerPlaying(playing : boolean) : void
+showBackInControl(player : Player) : void
+setTimerEnabled(enabled : boolean) : void
+showAutoPlay(player : Player) : void
+showQuestion(player : Player, question : Question, autoChoice : int, listener : Popups.Question.Listener) : void
+showSurprise(event : Game.SurpriseEvent, auto : boolean, listener : Popups.Surprise.Listener) : void
+showTimeWarning(player : Player) : void
+diceRolled(player : Player, roll : Dice.Roll) : void
+pieceMoved(result : Move.Result) : void
+noticeRaised(notice : Game.TurnNotice) : void
+turnStarted(player : Player, extraRoll : boolean) : void
+questionAsked(player : Player, question : Question) : void
+questionAnswered(answer : Game.Answer) : void
+surpriseDrawn(event : Game.SurpriseEvent) : void
+victimRequested(event : Game.SurpriseEvent) : void
+victimRolled(victim : Piece, stepRoll : int, steps : int) : void
+gameOver(ranking : List<Player>, duration : Duration) : void

BoardPanel
+MAX_MOVE_ANIMATION_MILLIS : int
-view : BoardPanel.Camera
-pieces : BoardPanel.Pieces
-animation : BoardPanel.Pieces.MoveAnimation
-effects : BoardPanel.Effects
-game : Game.State
-banner : BoardPanel.Banner
-effectsOn : boolean
-artGame : Game.State
-hover : Piece
-lastFrame : long
-pressed : boolean

+setGame(game : Game.State) : void
+setSelectablePieces(movable : Set<Piece>) : void
+setOnPieceClicked(handler : Consumer<Piece>) : void
+isAnimating() : boolean
+animateMove(move : Move, captured : List<Piece>, onDone : Runnable) : void
+comicBurst(text : String, caption : String, color : Color) : void
+comicBurst(text : String, caption : String, player : Player) : void
+showBanner(title : String, text : String, color : Color, millis : int, onClose : Runnable, icon : String) : void
+showBanner(title : String, text : String, color : Color, millis : int, onClose : Runnable, icon : String, speaker : Color) : void
+showBanner(title : String, text : String, color : Color, millis : int, onClose : Runnable) : void
+hideBanner() : void
+stopAnimation() : void
+setEffectsOn(on : boolean) : void
+getToolTipText(event : MouseEvent) : String

AdminLoginPanel
-creating : boolean
-revealed : boolean
-waitText : String
-waitSeconds : int

+setListener(listener : AdminLoginPanel.Listener) : void
+reset() : void
+setCreating(create : boolean) : void
+setBusy(busy : boolean) : void
+showFailure(waitSeconds : int) : void
+showLocked(waitSeconds : int) : void
+showTampered() : void
+showCreateProblems(problems : List<Security.Problem>) : void

AdminHomePanel
-shownQuestions : List<Question>
-countPerLevel : Map<Level, Long>

+setUser(username : String) : void
+showQuestions(questions : List<Question>) : void
+showBankProblem(reason : Reason) : void
+select(id : int) : void
+setListener(listener : AdminHomePanel.Listener) : void
+clear() : void

ProfilePanel
-revealed : boolean

+setListener(listener : ProfilePanel.Listener) : void
+reset(name : String) : void
+setBusy(busy : boolean) : void
+showResult(problems : List<Security.Problem>, name : String) : void

AppController
-uniqueInstance : AppController
-history : GameHistory
-timerEnabled : boolean
-effectsOn : boolean
-frame : MainFrame
-settings : SettingsPanel
-welcome : WelcomePanel
-menu : MenuPanel
-setup : SetupPanel
-gamePanel : GamePanel
-gameOverPanel : GameOverPanel
-historyPanel : HistoryPanel
-adminLogin : AdminLoginPanel
-adminHome : AdminHomePanel
-profile : ProfilePanel
-currentScreen : String
-screenBeforeSettings : String

+instance() : AppController
+main(args : String[]) : void
~openMenu() : void
~openMenuInstantly() : void
~showMenu() : void
~showWelcome() : void
~showAdminLogin() : void
~showAdminHome() : void
~showProfile() : void
~refreshSession() : void
~showGameRules() : void
+guestChosen() : void
+adminLoginChosen() : void
+exitChosen() : void
+logOutChosen() : void
+profileChosen() : void
+newGameChosen() : void
+historyChosen() : void
+questionsChosen() : void
+modeChanged(mode : Theme.Mode) : void
+languageChanged(language : Assets.Texts.Language) : void
+timerChanged(on : boolean) : void
+soundChanged(on : boolean) : void
+musicChanged(on : boolean) : void
+backdropChanged(backdrop : Theme.Backdrop) : void
+effectsChanged(on : boolean) : void
+rulesChosen() : void
+settingsClosed() : void
~startMatch(game : Game) : void
~matchFinished(game : Game, gameId : int) : void
~quitGameRequested() : void
+menuChosen() : void
+historyClosed() : void

SetupController
-uniqueInstance : SetupController
-panel : SetupPanel

+instance() : SetupController
~setScreen(panel : SetupPanel) : void
~open() : void
~createGame(level : Level, setups : List<Player.Setup>) : Game
+startChosen(level : Level, setups : List<Player.Setup>) : void
+setupClosed() : void
+setupProblems(setups : List<Player.Setup>) : List<Player.SetupProblem>
+isValidName(name : String) : boolean

GameController
+AUTO_MOVE_DELAY_MILLIS : int
-uniqueInstance : GameController
-game : Game
-gameId : int
-panel : GamePanel
-timerEnabled : boolean
-turnTimer : Game.TurnTimer
-rng : RandomGenerator
-lastTickMillis : long
-matchRunning : boolean
-paused : boolean
-autoPlaying : boolean

+instance() : GameController
~start(game : Game, gameId : int, panel : GamePanel, timerEnabled : boolean) : void
~quitConfirmed() : void
~moveToPanel(newPanel : GamePanel) : void
~setPaused(pause : boolean) : void
~stop() : void
~isRunning() : boolean
+rollRequested() : void
+pieceClicked(piece : Piece) : void
+rulesRequested() : void
+quitRequested() : void
+turnStarted(player : Player, extraRoll : boolean) : void
+diceRolled(player : Player, roll : Dice.Roll) : void
+pieceMoved(result : Move.Result) : void
+questionAsked(player : Player, question : Question) : void
+surpriseDrawn(event : Game.SurpriseEvent) : void
+answerChosen(index : int) : Game.Answer
+questionClosed() : void
+surpriseClosed() : void
+victimRequested(event : Game.SurpriseEvent) : void
+noticeClosed() : void
+noticeRaised(notice : Game.TurnNotice) : void
+gameOver(ranking : List<Player>, duration : Duration) : void

AdminController
-uniqueInstance : AdminController
-admin : Admin
-bank : QuestionBank
-loginPanel : AdminLoginPanel
-homePanel : AdminHomePanel
-profilePanel : ProfilePanel

+instance() : AdminController
~setScreens(loginPanel : AdminLoginPanel, homePanel : AdminHomePanel, profilePanel : ProfilePanel) : void
~openLogin() : void
~openHome(username : String) : void
~openProfile() : void
+loginRequested(username : String, password : char[]) : void
+accountCreationRequested(username : String, password : char[]) : void
+profileSaveRequested(username : String, current : char[], newPassword : char[], repeat : char[]) : void
+profileClosed() : void
+loginCancelled() : void
+menuRequested() : void
+logoutRequested() : void
+addRequested() : void
+editRequested(id : int) : void
+deleteRequested(id : int) : void

Settings
+FILE_NAME : String
+APPEARANCE : String
+LANGUAGE : String
+TURN_TIMER : String
+SOUND_EFFECTS : String
+MUSIC : String
+COMIC_EFFECTS : String
+BACKGROUND : String
-file : Path
-values : Properties

+load(folder : Path) : Settings
+contains(key : String) : boolean
+get(key : String, fallback : String) : String
+isOn(key : String) : boolean
+set(key : String, value : String) : void
+set(key : String, on : boolean) : void
+save() : boolean
+dataDirectory() : Path
+programDirectory() : Path

GameHistory
+FILE_NAME : String
+HEADER : String
-file : Path

+nextGameId() : int
+append(record : GameHistory.GameRecord) : void
+readAll() : GameHistory.ReadResult
~fromRow(line : String) : GameHistory.GameRecord
~toRow(record : GameHistory.GameRecord) : String

Game
+MAX_CONSECUTIVE_SIXES : int
+TURN_TIME_LIMIT_SECONDS : int
+TURN_WARNING_SECONDS : int
-level : Level
-state : Game.State
-board : Board
-players : List<Player>
-dice : Dice
-bonusDice : Dice
-questions : Question.Source
-surprises : Supplier<Surprise>
-clock : Clock
-startedAt : Instant
-listeners : List<Game.Listener>
-ranking : List<Player>
-skipNextTurn : Set<Player>
-firstPlayerIndex : int
-currentIndex : int
-round : int
-phase : Phase
-lastRoll : Dice.Roll
-lastRollWasBonus : boolean
-legalMoves : List<Move>
-consecutiveSixes : int
-bonusSixes : int
-regularRollOwed : boolean
-bonusRollsOwed : int
-notice : Game.TurnNotice
-question : Question
-lastAnswer : Game.Answer
-effectPiece : Piece
-surprise : Game.SurpriseEvent
-endedAt : Instant
-quitEarly : boolean
-stats : Game.Stats

+addListener(listener : Game.Listener) : void
+removeListener(listener : Game.Listener) : void
+roll() : Dice.Roll
+move(piece : Piece) : Move.Result
+answer(choice : int) : Game.Answer
+continueAfterAnswer() : void
+acknowledgeSurprise() : void
+chooseVictim(victim : Piece) : Move.Result
+victimChoices() : Set<Piece>
+acknowledgeNotice() : void
+onlyMove() : Optional<Piece>
+quit() : void
~askQuestion(piece : Piece, level : Level) : void
~drawSurprise(piece : Piece) : void
~endAction() : void
+wasQuitEarly() : boolean
+winner() : Optional<Player>
+currentPlayer() : Player
+isOver() : boolean
+nextRollIsBonus() : boolean
+movablePieces() : Set<Piece>
+losesNextTurn(player : Player) : boolean
+elapsed() : Duration

Admin
+DEFAULT_USERNAME : String
-accountFile : Path
-throttleFile : Path
-iterations : int
-random : SecureRandom
-clock : Clock
-role : Role
-username : String

+enterAsGuest() : void
+isAdmin() : boolean
+hasAccount() : boolean
+ensureDefaultAccount() : boolean
+login(username : String, password : char[]) : Admin.LoginResult
+changePassword(current : char[], newPassword : char[], repeat : char[]) : List<Security.Problem>
+updateProfile(current : char[], newUsername : String, newPassword : char[], repeat : char[]) : List<Security.Problem>
+createAccount(username : String, password : char[], repeat : char[]) : List<Security.Problem>

QuestionBank
+MIN_TOTAL : int
+MIN_PER_LEVEL : int
-bankFile : Path
-keyFile : Path
-random : SecureRandom
-uniqueInstance : QuestionBank
-questions : List<Question>
-loadProblem : Reason

+instance() : QuestionBank
+reload() : void
+isAvailable() : boolean
+canPlay(level : Level) : boolean
+count(level : Level) : int
+newDeck(rng : RandomGenerator) : Question.Deck
+add(draft : Question) : Question
+update(changed : Question) : void
+deleteProblem(id : int) : QuestionBank.DeleteCheck
+delete(id : int) : void
~toFileText(questions : Collection<Question>) : String
~fromFileText(text : String) : List<Question>

Player
+MIN_PLAYERS : int
+MAX_PLAYERS : int
+PIECES_PER_PLAYER : int
+NAME_MIN_LENGTH : int
+NAME_MAX_LENGTH : int
-name : String
-color : Player.Color
-seat : Seat
-pieces : List<Piece>

+finishedCount() : int
+hasFinishedAll() : boolean
+setupMessages(setups : List<Player.Setup>) : List<String>
+setupProblems(setups : List<Player.Setup>) : List<Player.SetupProblem>
+isValidName(name : String) : boolean

Move
-piece : Piece
-kind : Kind
-from : int
-to : int

+destination() : Square
+changesPosition() : boolean
+traversedSquares() : List<Square>
+legalMoves(player : Player, roll : Dice.Roll, players : List<Player>) : List<Move>
+piecesOn(square : Square, players : List<Player>) : List<Piece>
+hasOpponentBlock(square : Square, mover : Player, players : List<Player>) : boolean
+capturedBy(mover : Piece, players : List<Player>) : List<Piece>
+progressAfterEffect(piece : Piece, steps : int, players : List<Player>) : int
+piecesThatCanGoBack(player : Player, players : List<Player>) : List<Piece>
+whyNoMove(player : Player, roll : Dice.Roll, players : List<Player>) : Set<Move.NoMoveReason>
+chooseMove(moves : List<Move>, players : List<Player>) : Move
+chooseVictim(victims : List<Piece>) : Piece

«interface» Dice
+STEP_DIE_FACES : int
+RELEASE_ROLL : int
+ULTRA_BOOST_FACES : int

+roll() : Dice.Roll
+forLevel(level : Level, rng : RandomGenerator) : Dice

«enumeration» Surprise
JUMP_TO_FINISH
OPPONENT_BACK
BACK_TO_CASTLE
BACK_THREE
LOSE_NEXT_TURN
EXTRA_ROLL
BACK_STEPS

«enumeration» Role
GUEST
ADMIN

Security
+FAILURE_MESSAGE : String
+PASSWORD_MIN : int
+PASSWORD_MAX : int
+USERNAME_MIN : int
+USERNAME_MAX : int
+HASH_ITERATIONS : int

+passwordProblems(password : char[], confirmation : char[]) : List<Security.Problem>
+usernameProblems(username : String) : List<Security.Problem>
+hash(password : char[], iterations : int, random : SecureRandom) : Security.PasswordHash
+matches(password : char[], stored : Security.PasswordHash) : boolean
+hashLogin(username : String, password : char[], iterations : int, random : SecureRandom) : Security.PasswordHash
+loginMatches(username : String, password : char[], stored : Security.PasswordHash) : boolean
+normalizeUsername(username : String) : String
+usernameTag(username : String, salt : byte[]) : byte[]
+warmUpHashing() : void
~sealKey(purpose : String) : byte[]
+seal(purpose : String, values : String[]) : String
+sealMatches(seal : String, purpose : String, values : String[]) : boolean
+encrypt(plain : byte[], key : SecretKey, header : byte[], random : SecureRandom) : byte[]
+decrypt(data : byte[], key : SecretKey, header : byte[]) : Optional<byte[]>
+hashHeader(data : byte[], header : byte[]) : boolean
+readKey(keyFile : Path) : Optional<SecretKey>
+createKey(keyFile : Path, random : SecureRandom) : SecretKey
+readProperties(file : Path) : Properties
+writeProperties(file : Path, p : Properties, comment : String) : void
+writeBytes(file : Path, data : byte[]) : void

Question
-id : int
-text : String
-answers : List<String>
-correctIndex : int
-explanation : String
-level : Level
-topic : Topic
+TEXT_MIN : int
+TEXT_MAX : int
+ANSWER_COUNT : int
+ANSWER_MAX : int
+EXPLANATION_MAX : int

+withId(newId : int) : Question
+correctAnswer() : String
+steps() : int
+problems(text : String, answers : List<String>, correct : int, explanation : String, level : Level, topic : Topic) : List<Question.Problem>
+computerAnswer(question : Question, rng : RandomGenerator) : int

«enumeration» Reason
KEY_MISSING
CHANGED
DAMAGED

«enumeration» Phase
AWAITING_ROLL
AWAITING_MOVE
AWAITING_ANSWER
SHOWING_ANSWER
SHOWING_SURPRISE
CHOOSING_VICTIM
SHOWING_NOTICE
GAME_OVER

Piece
+IN_CASTLE : int
-owner : Player
-number : int
-path : List<Square>
-progress : int

+lastIndex() : int
+currentSquare() : Square
+isInCastle() : boolean
+isFinished() : boolean
+isOnBoard() : boolean
+isInHomeLane() : boolean
~setProgress(progress : int) : void
~sendToCastle() : void

«enumeration» Kind
RELEASE
FULL
STOP_BEFORE_BLOCK
QUESTION_FORWARD
QUESTION_BACK
JUMP_TO_FINISH
SURPRISE_BACK
TO_CASTLE

Boosted
-rng : RandomGenerator
-multiplierFaces : int

+roll() : Dice.Roll

Plain
-rng : RandomGenerator

+roll() : Dice.Roll

Board
+HOME_LANE_LENGTH : int
-level : Level
-rows : int
-cols : int
-sharedTrack : List<Square>
-squares : List<Square>
-paths : Map<Seat, List<Square>>
-castles : Map<Seat, Board.GridArea>
-center : Board.GridArea

+pathFor(seat : Seat) : List<Square>
+startSquare(seat : Seat) : Square
+castleArea(seat : Seat) : Board.GridArea
+centerArea() : Board.GridArea
+countSquaresOfType(type : Type) : long
+build(level : Level, playing : Collection<Seat>, rng : RandomGenerator) : Board
+build(level : Level, rng : RandomGenerator) : Board

«enumeration» Topic
AGILE
MVC
DESIGN_PATTERNS
QUALITY_ASSURANCE
SOFTWARE_DESIGN

Square
-id : int
-type : Type
-position : Board.GridPosition
-owner : Seat

+create(id : int, type : Type, pos : Board.GridPosition, owner : Seat, questionLevel : Level) : Square
~land(game : Game, piece : Piece) : void
+questionLevel() : Level
+isProtected() : boolean
+isArrowFor(seat : Seat) : boolean

QuestionSquare
-level : Level

~land(game : Game, piece : Piece) : void
+questionLevel() : Level

SurpriseSquare
~land(game : Game, piece : Piece) : void

«enumeration» Type
REGULAR
STAR
START
ARROW
HOME_LANE
FINISH
QUESTION
SURPRISE

«enumeration» Seat
TOP_LEFT
TOP_RIGHT
BOTTOM_RIGHT
BOTTOM_LEFT

«enumeration» Level
EASY
MEDIUM
HARD